TEACHING LAB / SKY130A / TINY TAPEOUT ANALOG

From CMOS Schematic to Verified GDS

A step-by-step student guide to Xschem, ngspice, Magic, extraction, LVS, parasitics, the Tiny Tapeout 1×2 wrapper, LEF/GDS export, and KLayout inspection.

CASE STUDY	PROCESS	SCOPE
1.8 V CMOS inverter	SKY130A / 130 nm	Local verification; no upload
2 transistors	PMOS 2.0/0.15 µm; NMOS 1.0/0.15 µm	1×2 analog tile, two ua pins



Based on the working lab session: DRC-clean inverter, standalone LVS match, pre-/post-layout simulation, verified Tiny Tapeout port connectivity, correct output grounding, GDS export and independent GDS re-import checks.

> **Scope and honesty**
>
> The exact **centered wrapper** is our verified teaching baseline. A shorter-route, lower-right placement is included later as a **proposed extension**, not as a completed or tapeout-validated implementation. Official Tiny Tapeout precheck and upload were not performed.

Prepared for introductory analog IC design labs | November 2026

How to use this guide

Read the concepts first; execute each numbered command block in the place printed above it. **Linux shell** means the normal terminal prompt. **Magic Tkcon** means the separate Tcl console beginning with %. Never paste shell commands into Tkcon or vice versa.

This manual follows the actual completed inverter workflow, not a generic OpenLane digital flow. It includes pictures from the lab session and illustrated floorplans for teaching.

> **Copying commands**
>
> Commands containing paths such as `/foss/designs/...` assume the IIC-OSIC-TOOLS container and its mounts. Device-cell filenames are generated by the PDK tool and may differ on a fresh machine; confirm them with `ls` before copying.

Four working habits that prevent most failures

Habit	Why it matters
Save after each successful stage	Magic source edits persist only when written to the correct .mag cell.
Confirm the active directory	Extraction into mag/ is not the same evidence as extraction into gds_check/.
Set a fresh, tiny box before a via	A stale oversized via box can create widespread physical shorts.
Run DRC and extraction at checkpoints	One discovered problem is simpler to fix than 20 accumulated problems.

What to keep for the lab report

Save a schematic screenshot, a pre-layout waveform, the standalone DRC/LVS evidence, the extracted transistor-level SPICE result, a complete Tiny Tapeout top-cell view, the GDS-derived inverter instance line, and the final LEF/GDS files. These show how each stage was validated, not just what buttons were clicked.

Contents

01 The complete analog design roadmap 4

02 Know the tools and the two command windows 5

03 Inverter schematic and pre-layout simulation 6

04 Create and inspect the standalone Magic layout 8

05 DRC, extraction, LVS and PEX of the standalone inverter 10

06 Create the Tiny Tapeout 1×2 wrapper 11

07 Power rails and original inverter placement 13

08 Route VDPWR and VGND using Metal3 14

09 Connect ua[0] and ua[1] to the inverter 15

10 Ground unused Tiny Tapeout digital outputs 17

11 Final DRC: how we fixed real layout errors 19

12 Generate final LEF and GDS 20

13 KLayout and independent GDS round-trip inspection 21

14 Placement matters: a proposed optimization lab 23

15 Troubleshooting reference 24

16 Evidence-based verification checklist 25

Appendix A Quick command card 25

Appendix B Source material and references 26

Useful principle: **DRC asks whether the geometry is legal; LVS asks whether it represents the intended circuit; extraction reveals actual connectivity; simulation measures behavior; KLayout lets us inspect the final physical database.** None of these alone is a substitute for the others.

01 The complete analog design roadmap

A standard-cell digital design starts from RTL synthesis and automatic place-and-route. This lab is different: the transistors are chosen and arranged explicitly, and we route a custom analog block by hand.



Stage	Main question	Evidence
Xschem schematic	Is the intended circuit connected correctly?	Two MOSFETs, explicit VIN/VOUT/VDD/GND
Pre-layout SPICE	Does it invert 0–1.8 V?	Waveform and timing measurements
Standalone Magic layout	What transistor and wire geometry did we draw?	Hierarchical .mag cells
DRC / LVS	Manufacturable and electrically equivalent?	No DRC violations; Netgen unique match
PEX / post-layout SPICE	How do physical parasitics change delay?	Comparative transient analysis
Tiny Tapeout wrapper	Does it connect to valid physical chip pins?	ua[0], ua[1], VDPWR, VGND
Final GDS + LEF	What data is delivered to integration?	Geometric GDS + abstract LEF
KLayout / GDS round-trip	Did export preserve the intended layout?	Visual hierarchy + imported-GDS DRC/extraction

What counts as finished locally

We finished through GDS generation and local GDS verification. **Independent full-wrapper LVS, official Tiny Tapeout precheck and shuttle submission remain separate activities.**

02 Know the tools and the two command windows

2.1 Tool responsibilities

Tool	Purpose	Typical file
Xschem	Capture the inverter schematic	.sch
ngspice	Simulate transient waveforms and delay	.spice / plots
Magic	Draw, inspect and DRC custom transistor layouts	.mag / .ext
Netgen	Compare extracted layout against schematic netlist	LVS report
KLayout	Inspect GDS hierarchy/layers; optionally run verification	.gds
LEF	Abstract block size, access pins and obstructions	.lef
GDSII	Fabrication geometry and cell hierarchy	.gds

2.2 Linux shell versus Magic Tkcon

A shell prompt in the container looks like `/foss/designs/tt_inverter >`. Use it for filesystem operations, launching GUIs, reading files and searching extracted netlists. Tkcon begins with `%`; use it for Magic commands that manipulate layout geometry.

LINUX TERMINAL $

```
cd /foss/designs/tt_inverter/mag
ls -lh
magic -rcfile /foss/pdks/sky130A/libs.tech/magic/sky130A.magicrc
```

MAGIC TKCON %

```
box values 80um 100um 80um 100um
getcell inv_layout_clean
expand all
save
```

Critical Magic habit

Before each `sky130::via*_draw` call, explicitly set a **small** box with `box values ....` A stale huge box once produced accidental power shorts in this lab.

2.3 The project folders we actually used

FOLDER MAP / NOT A COMMAND

```
/foss/designs/inverter/
    mag/inv_layout_clean.mag        # verified standalone core
    mag/sky130_fd_pr__*.mag         # generated device cells
/foss/designs/tt_inverter/
    mag/tt_analog_1x2.def           # physical interface template
    mag/tt_um_panteloukos_inverter.mag
    info.yaml
    src/project.v
    gds/tt_um_panteloukos_inverter.gds
    lef/tt_um_panteloukos_inverter.lef
    gds_check/                     # separate round-trip workspace
```

The generated analog DEF comes from the official Tiny Tapeout support-tools repository. The inverter was copied as a hierarchy of .mag files, not redrawn from scratch during the wrapper phase.

03 Inverter schematic and pre-layout simulation

3.1 Understand the circuit before touching layout

The CMOS inverter has a PMOS pull-up between VDD and VOUT and an NMOS pull-down between VOUT and GND. Both gates connect to VIN. PMOS bulk must connect to VDD and NMOS bulk to GND. Logic-high input turns on NMOS; logic-low turns on PMOS.

Device	Type / dimensions	Schematic connections
MP0	sky130 pfet_01v8; W=2.0 µm; L=0.15 µm	D=VOUT, G=VIN, S=VDD, B=VDD
MN0	sky130 nfet_01v8; W=1.0 µm; L=0.15 µm	D=VOUT, G=VIN, S=GND, B=GND

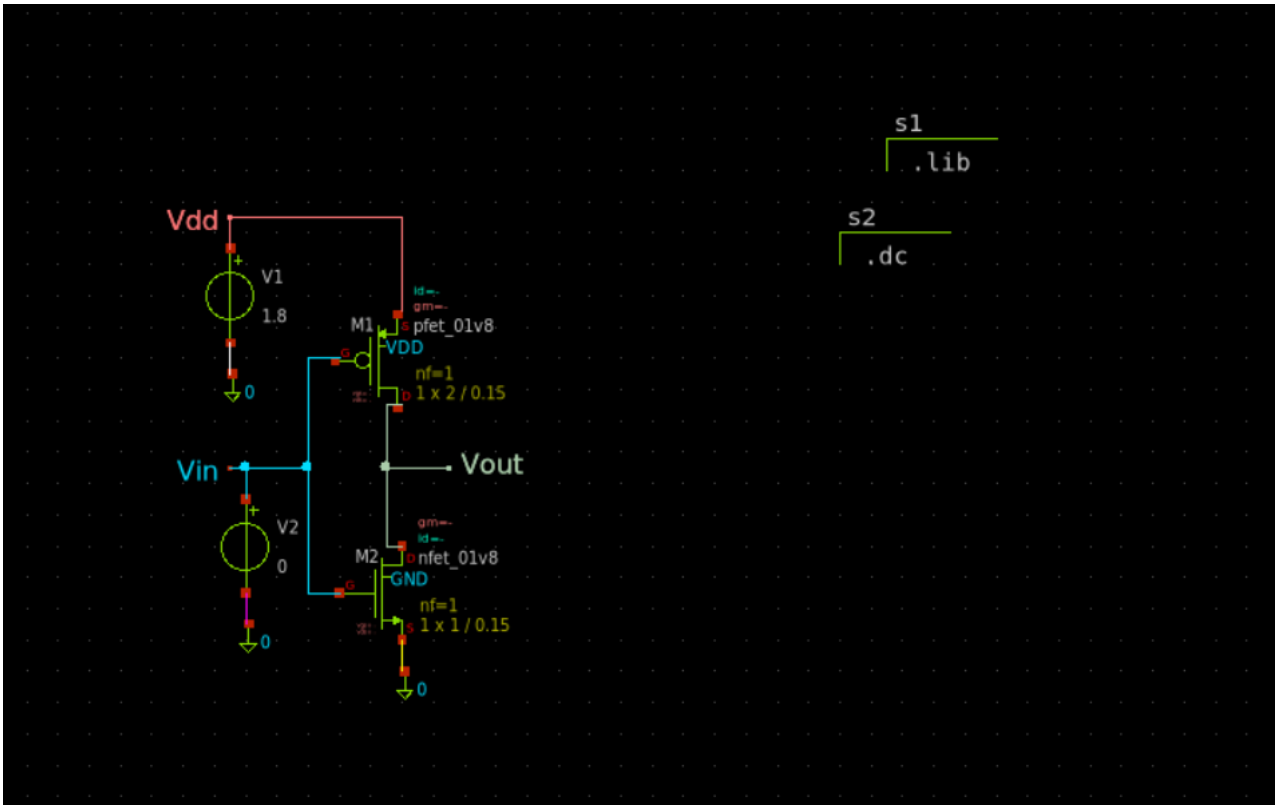


Figure 1 — The original Xschem teaching schematic, with VIN on both gates and VOUT at the common drain node.

3.2 Build the Xschem testbench

GUI steps: Open Xschem in the SKY130 environment. Place one PDK pfet_01v8 and one nfet_01v8 symbol. Set W=2, L=0.15 for PMOS and W=1, L=0.15 for NMOS; use one finger and multiplicity one. Wire gates, drains, sources, and bulks as above. Add VDD=1.8 V, a VIN pulse source, and a 10 fF load from VOUT to ground.

LINUX TERMINAL $ (REPLACE PLACEHOLDER)

```
cd /foss/designs/inverter
find . -type f -name "*.sch" -print
# Open the actual schematic path reported by find:
xschem /foss/designs/inverter/<actual-name>.sch
```

Use an ngspice voltage source equivalent to the following pulse; check the exact Xschem symbol attributes in your installed setup.

NGSPICE TESTBENCH / ILLUSTRATIVE SPICE LINES

```
VIN VIN 0 PULSE(0 1.8 1n 100p 100p 10n 20n)
CLOAD VOUT 0 10f
.tran 10p 100n
```

Run Xschem Netlist and Simulate. Plot VIN and VOUT on the same time axis. A correct inverter produces complementary square waves. Delay is normally measured at 50% VDD = 0.9 V crossings, with opposite input/output edges.

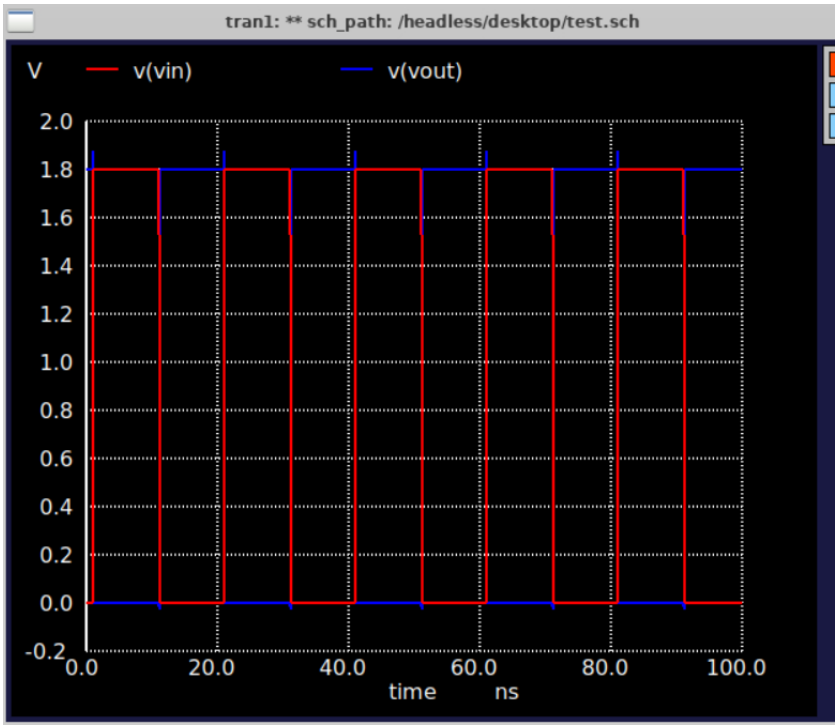


Figure 2 — Pre-layout inverter transient response from the lab: VIN and VOUT switch in opposite directions.

Measurement	Recorded result	Meaning
tPHL	about 48.85 ps	VOUT high-to-low after VIN rises
tPLH	about 57.66 ps	VOUT low-to-high after VIN falls
VDD	1.8 V	Supply for this lab
Output load	10 fF	Pre-layout test load

> **Do not confuse schematic and layout timing**
>
> These values were recorded for the **standalone inverter schematic**. They are not guaranteed timing numbers for the full, later Tiny Tapeout wrapper with long analog pin routing.

04 Create and inspect the standalone Magic layout

4.1 Start Magic with the correct PDK

LINUX TERMINAL $

```
cd /foss/designs/inverter/mag
magic -rcfile /foss/pdks/sky130A/libs.tech/magic/sky130A.magicrc
```

The `-rcfile` option loads SKY130 technology rules, layers, device generators and DRC definitions. Do not use an IHP PDK session for SKY130 cells.

4.2 Place devices and draw their local connections

For a new standalone lab, instantiate the SKY130 1.8 V PMOS and NMOS PCells using Magic's SKY130 device generator. Set the same sizes as the schematic, put PMOS above NMOS, keep adequate room for terminal contacts, then connect their gates (VIN), drains (VOUT), PMOS source/body (VDD), and NMOS source/body (GND). This is an interactive cell-generation step: exact PCell dialog fields and generated cell suffixes depend on the installed PDK.

For **our fully reproducible continuation**, do not regenerate the transistors. Open the existing tested cell and its PCell children:

MAGIC TKCON %

```
load inv_layout_clean
expand all
box
```

If the geometry appears as a narrow outline rather than transistors, move the mouse into the correct Magic layout window and use `expand all`. This only changes display expansion. Never use `flatten` to "make the devices visible".

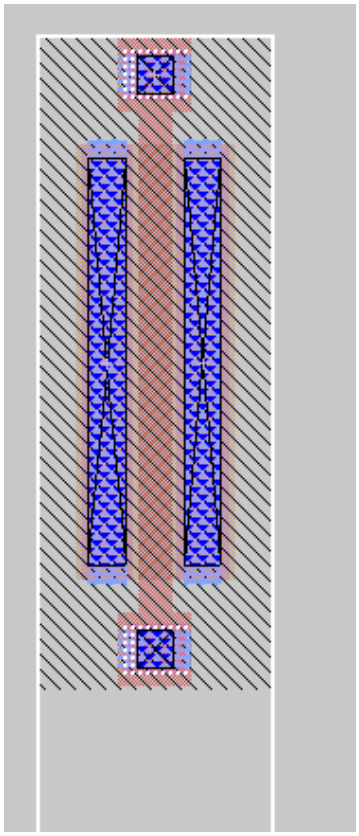


Figure 3 — Close-up of the handcrafted inverter geometry. The tall local structure is normal for the very small device count.

4.3 Core routing and ports

Metal1 is convenient for local transistor wiring. Metal2 is used later for longer signals; Metal3 for wrapper-level power; Metal4 belongs to Tiny Tapeout pin access. Use a tight box, paint the correct layer, and inspect what it touches before assuming that two polygons are electrically connected.

MAGIC TKCON % (INSPECTION DEMO; NO PAINT)

```
box values 80um 100um 81um 100.30um
# Example only: do NOT paint this example into a real cell.
# box sets a physical rectangle; paint metal1 would fill it.
box
select clear
select area
what
```

For the original standalone inverter, we ended with four named metal1 ports: `VIN, VOUT, VDD, GND`. You can inspect the verified cell using selections near its terminals rather than recreating the labels.

MAGIC TKCON %

```
load inv_layout_clean
expand all
save
```

> **Why cell hierarchy matters**
>
> The standalone `inv_layout_clean.mag` contains instances of the generated PMOS and NMOS .mag cells. When copying it into the Tiny Tapeout project, all referenced device .mag files must also be available.

05 DRC, extraction, LVS and PEX of the standalone inverter

5.1 Design Rule Check: can the geometry be fabricated?

MAGIC TKCON %

```
drc check
drc catchup
drc find 1
```

A clean result reads `There are no errors in inv_layout_clean.` If not, use `drc why` followed by box to read the error class and exact coordinates. Fix geometry, save, and rerun DRC.

MAGIC TKCON % (AFTER AN ERROR)

```
drc why
box
select clear
select area
what
```

What DRC does not prove: a short between VIN and VOUT could still obey all spacing/width rules. For functionality we need extraction and LVS.

5.2 Extract the layout as a SPICE netlist

MAGIC TKCON %

```
extract all
ext2spice lvs
ext2spice
```

Magic follows connected polygons through device contacts, metal and vias; the extracted cell netlist shows what was physically built. With this simplified LVS mode, the geometry can be compared to Xschem's reference netlist using Netgen.

5.3 Netgen LVS: compare layout with schematic

Generate a schematic SPICE netlist in Xschem. Identify the actual top-cell name in both netlists before invoking Netgen. The command below is a **template** (not a claim that these filenames exist in a fresh lab):

LINUX TERMINAL $ (FILL IN THE REAL FILENAMES)

```
netgen -batch lvs \
  "<extracted-layout.spice> inv_layout_clean" \
  "<xschem-reference.spice> <schematic-top-name>" \
  /foss/pdks/sky130A/libs.tech/netgen/sky130A_setup.tcl \
  lvs_report.txt
```

A successful comparison in our standalone test ended with `Circuits match uniquely.` This is stronger than simply seeing the same number of transistors: it verifies device types and matched electrical connectivity under the LVS comparison setup.

5.4 Post-layout parasitic simulation

Make a separate backup of the LVS netlist, then request extracted parasitic capacitances (and the available resistance information according to Magic extraction settings). Use the resulting SPICE with the same 1.8 V supply and pulse/load testbench to compare pre- and post-layout behavior.

MAGIC TKCON % (PARASITIC EXTRACTION)

```
extract all
ext2spice cthresh 0
ext2spice rthresh 0
ext2spice
```

Metric	Pre-layout	Post-layout	Change
tPHL	48.85 ps	52.76 ps	about +8.0%
tPLH	57.66 ps	62.46 ps	about +8.3%

The lesson is concrete: wires and device geometry add parasitics and slow the inverter. These results were measured for the standalone inverter before embedding it into the Tiny Tapeout wrapper.

06 Create the Tiny Tapeout 1×2 wrapper

6.1 Check the official interface rules

This walkthrough uses the **SKY130A 1×2** analog DEF. The imported geometry in our session was exactly **161.000 µm × 225.760 µm**. The public spec gives rounded tile dimensions; **the actual imported DEF/LEF coordinates govern**. The design uses only 1.8 V VDPWR, no VAPWR, and two analog pins.

Pin	Purpose	Expected region
ua[0]	VIN	bottom metal4, x 151.81–152.71 µm
ua[1]	VOUT	bottom metal4, x 132.49–133.39 µm
VDPWR	1.8 V	left metal4 x 0–2 µm
VGND	ground	left metal4 x 3–5 µm

Keep every Tiny Tapeout template pin size and location unchanged. The interface includes top-edge digital pins even when the actual analog circuit only uses the two ua pins. User routing must not use Metal5 or via4.

6.2 Prepare local copies (no upload)

LINUX TERMINAL $ (ONE-TIME SETUP)

```
cd /foss/designs
# If not already cloned, make local copies:
git clone https://github.com/TinyTapeout/tt-support-tools.git
git clone https://github.com/TinyTapeout/ttsky-analog-template.git tt_inverter
mkdir -p tt_inverter/mag tt_inverter/gds tt_inverter/lef
cp tt-support-tools/tech/sky130A/def/analog/tt_analog_1x2.def \
   tt_inverter/mag/
```

A `git clone` downloads a public copy. It does **not** push your work anywhere. If you already have `/foss/designs/tt_inverter`, skip cloning and check the existing files.

LINUX TERMINAL $ (COPY OUR VERIFIED HIERARCHY)

```
cd /foss/designs/tt_inverter/mag
cp /foss/designs/inverter/mag/inv_layout_clean.mag .
cp /foss/designs/inverter/mag/sky130_fd_pr__pfet_01v8_YR6QPW.mag .
cp /foss/designs/inverter/mag/sky130_fd_pr__nfet_01v8_Y5M242.mag .
ls -lh *.mag tt_analog_1x2.def
```

6.3 Import the DEF into Magic

LINUX TERMINAL $

```
cd /foss/designs/tt_inverter/mag
magic -rcfile /foss/pdks/sky130A/libs.tech/magic/sky130A.magicrc
```

MAGIC TKCON %

```
def read tt_analog_1x2.def
save tt_um_panteloukos_inverter
```

The DEF defines the fixed die boundary and top/bottom pin geometry. A successful import reports many pins; the `At minimum scale!` messages seen in our import were nonfatal. Do not assume the DEF special-net naming alone defines the final power pin; verify the final LEF and extraction.

07 Power rails and original inverter placement

7.1 Draw the left-side Metal4 power pins

We used two 2 µm-wide Metal4 stripes from y=5 to y=220.76 µm. They satisfy the published rule of at least 1.2 µm width and running near both the bottom and top edges. Create labels and declare their port uses. The following names are referenced explicitly so Magic uses the correct selected port:

MAGIC TKCON % (VDPWR)

```
box values 0um 5um 2um 220.76um
paint metal4
label VDPWR
port make
port VDPWR class bidirectional
port VDPWR use power
```

MAGIC TKCON % (VGND)

```
box values 3um 5um 5um 220.76um
paint metal4
label VGND
port make
port VGND class bidirectional
port VGND use ground
save
```

Verify instead of trusting labels

The labels must be attached to the Metal4 rectangles. At the end, the LEF must show `PIN VDPWR ... USE POWER` and `PIN VGND ... USE GROUND`. If not, inspect the selected label/port and correct it before exporting.

7.2 Instantiate the verified inverter in the center (baseline)

MAGIC TKCON %

```
box values 80um 100um 80um 100um
getcell inv_layout_clean
expand all
save
```

The actual wrapped child was `inv_layout_clean_0`. Because instance origins depend on the child cell's own internal coordinates, we did not guess its terminals. We located them using `box values`, `select area` and `what`.

Terminal	Measured wrapper coordinate range
VIN	x ≈ 80.54–80.68, y ≈ 105.02–105.22 µm
VOUT	x ≈ 80.02–80.18, y ≈ 104.92–105.22 µm
VDD	x ≈ 81.55–82.35, y ≈ 111.37–111.57 µm
GND	x ≈ 81.55–82.35, y ≈ 100.77–100.97 µm

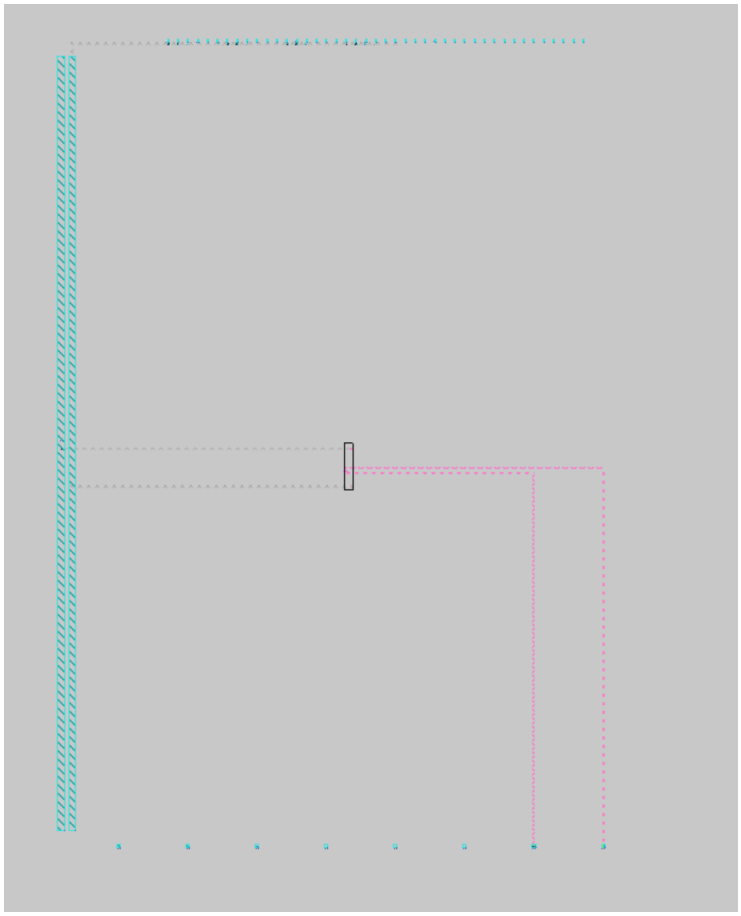


Figure 4 — Our original top-level Magic layout: left power rails, the tiny inverter around the middle, and long analog routes from the bottom. Large empty area is expected.

7.3 Why this is a baseline, not ideal floorplanning

Center placement made the routing easy to reason about while learning hierarchy and DRC, but it forced long analog nets. We kept it because it was fully verified. Section 14 describes how students can build and validate a better pin-aware alternative.

08 Route VDPWR and VGND using Metal3

A via bridges adjacent metal layers: via1 is M1–M2, via2 is M2–M3, and via3 is M3–M4. We route power horizontally on Metal3, then convert to Metal1 at the inverter terminal and to Metal4 at the outer rail.

8.1 VDPWR to the PMOS VDD terminal

MAGIC TKCON % (LOCAL VIA STACKS)

```
box values 81.75um 111.27um 82.15um 111.67um
sky130::via1_draw
box values 81.75um 111.27um 82.15um 111.67um
sky130::via2_draw
box values 0.75um 111.22um 1.25um 111.72um
sky130::via3_draw
```

MAGIC TKCON % (VDPWR HORIZONTAL METAL3)

```
box values 1.00um 111.27um 82.00um 111.67um
paint metal3
save
```

8.2 VGND to the NMOS GND terminal

MAGIC TKCON % (LOCAL VIA STACKS)

```
box values 81.75um 100.67um 82.15um 101.07um
sky130::via1_draw
box values 81.75um 100.67um 82.15um 101.07um
sky130::via2_draw
box values 3.75um 100.62um 4.25um 101.12um
sky130::via3_draw
```

MAGIC TKCON % (VGND HORIZONTAL METAL3)

```
box values 4.00um 100.67um 82.00um 101.07um
paint metal3
save
```

These exact boxes were used successfully in the **centered, original** wrapper. They must not be copied into a newly relocated wrapper unchanged: every inverter-side via coordinate changes when you move the cell.

8.3 Extract after power routing

MAGIC TKCON %

```
extract all
ext2spice lvs
ext2spice
```

LINUX TERMINAL $

```
cd /foss/designs/tt_inverter/mag
grep -n -A1 "Xinv_layout_clean" tt_um_panteloukos_inverter.spice
```

At this checkpoint, the child's last two nodes must be `VDPWR VGND`; VIN and VOUT may still have instance-local names because they are not routed yet. Stop if VDD and GND collapse onto the same node.

Stop-on-short rule

Do not continue routing after an extraction shows `VDD = VGND` or a merged signal. In our initial failed power attempt, a large accidentally retained box caused extensive via3 material and a supply short. We rebuilt from the clean DEF before continuing.

09 Connect ua[0] and ua[1] to the inverter

We used Metal2 for long signals so they could cross Metal3 power routes **without electrical contact**. Only a deliberately placed via connects metal layers. The two analog-pin stacks each descend Metal4 → via3 → Metal3 → via2 → Metal2.

9.1 Route ua[0] to VIN

MAGIC TKCON % (ua[0] VIA STACK + M3 PAD)

```
box values 152.06um 0.30um 152.46um 0.70um
sky130::via3_draw
box values 152.06um 0.30um 152.46um 0.70um
sky130::via2_draw
box values 152.01um 0.25um 152.51um 0.75um
paint metal3
```

MAGIC TKCON % (VIN METAL2 ROUTING)

```
box values 152.11um 0.50um 152.41um 106.10um
paint metal2
box values 80.60um 105.80um 152.41um 106.10um
paint metal2
box values 80.50um 105.10um 80.80um 106.10um
paint metal2
```

Before the VIN via, inspect overlap with the inverter’s labeled Metal1. We originally had a Metal1 spacing issue at the adjacent VIN/VOUT vias; the corrected VIN via below is offset to the right.

MAGIC TKCON % (VIN M1–M2 VIA; CORRECTED)

```
box values 80.50um 104.98um 80.80um 105.28um
select clear
select area
what
select clear
box values 80.50um 104.98um 80.80um 105.28um
sky130::via1_draw
save
```

9.2 Route ua[1] to VOUT on a separate lower track

MAGIC TKCON % (ua[1] VIA STACK + M3 PAD)

```
box values 132.74um 0.30um 133.14um 0.70um
sky130::via3_draw
box values 132.74um 0.30um 133.14um 0.70um
sky130::via2_draw
box values 132.69um 0.25um 133.19um 0.75um
paint metal3
```

MAGIC TKCON % (VOUT METAL2 ROUTING)

```
box values 132.79um 0.50um 133.09um 104.60um
paint metal2
box values 80.05um 104.30um 133.09um 104.60um
paint metal2
box values 79.95um 104.30um 80.25um 105.15um
paint metal2
```

MAGIC TKCON % (VOUT M1–M2 VIA)

```
box values 79.95um 104.90um 80.25um 105.20um
select clear
select area
what
select clear
box values 79.95um 104.90um 80.25um 105.20um
sky130::via1_draw
save
```

9.3 Verify the electrical net names, not just the colored lines

MAGIC TKCON %

```
extract all
ext2spice lvs
ext2spice
```

LINUX TERMINAL $

```
cd /foss/designs/tt_inverter/mag
grep -n -A1 -i "Xinv_layout_clean" \
  tt_um_panteloukos_inverter.spice
```

EXPECTED EXTRACTED SPICE / NOT A COMMAND

```
Xinv_layout_clean_0 ua[0] ua[1] VDPWR VGND inv_layout_clean
```

That one line proves the parent-side connection order: VIN=ua[0], VOUT=ua[1], VDD=VDPWR and GND=VGND, because `inv_layout_clean` defines its ports in that order. It does not replace the separate standalone device-level LVS test.

10 Ground unused Tiny Tapeout digital outputs

10.1 Which ports must not float?

The official SKY130 analog specification says unused `uo_out[7:0]`, `uio_out[7:0]` and `uio_oe[7:0]` must not float. That is 24 output-type pin shapes on the top edge. The input-only pins may remain unused. The `ua[2]`–`ua[5]` analog ports were not purchased/used in this exercise.

10.2 Create a Metal3 common ground bus near the top

MAGIC TKCON % (COMMON GROUNDED BUS)

```
box values 3.75um 220.20um 4.25um 224.65um
paint metal3
box values 4.00um 224.15um 94.55um 224.65um
paint metal3
box values 3.80um 220.20um 4.20um 220.60um
sky130::via3_draw
save
```

The trunk joins the existing Metal4 VGND rail, while the horizontal Metal3 line runs underneath the relevant top-edge Metal4 pads.

10.3 Ground one test pin before the whole bus

MAGIC TKCON % (FIRST: uio_oe[7])

```
box values 30.57um 224.15um 31.07um 224.65um
paint metal4
box values 30.67um 224.60um 30.97um 224.80um
paint metal4
box values 30.62um 224.20um 31.02um 224.60um
sky130::via3_draw
save
```

Run DRC and extraction now. The warning `Ports "VGND" and "uio_oe[7]" are electrically shorted.` is **intentional** in this particular case. It tells us that the top-edge output port was merged with the ground net.

10.4 Repeat the proven geometry for the other 23 pins

This loop uses the final corrected physical centers and exact pin edges from the imported DEF. The first pin, `uio_oe[7]` at x=30.82 µm, is deliberately excluded because we already connected it.

MAGIC TKCON % (USE ONLY WITH THIS EXACT DEF)

```
foreach {c xl xr} {
  33.58 33.43 33.73   36.34 36.19 36.49
  39.10 38.95 39.25   41.86 41.71 42.01
  44.62 44.47 44.77   47.38 47.23 47.53
  50.14 49.99 50.29   52.90 52.75 53.05
  55.66 55.51 55.81   58.42 58.27 58.57
  61.18 61.03 61.33   63.94 63.79 64.09
  66.70 66.55 66.85   69.46 69.31 69.61
  72.22 72.07 72.37   74.98 74.83 75.13
  77.74 77.59 77.89   80.50 80.35 80.65
  83.26 83.11 83.41   86.02 85.87 86.17
  88.78 88.63 88.93   91.54 91.39 91.69
  94.30 94.15 94.45
} {
  box values [expr {$c-0.25}]um 224.15um \
             [expr {$c+0.25}]um 224.65um
  paint metal4
  box values ${xl}um 224.60um ${xr}um 224.80um
  paint metal4
  box values [expr {$c-0.20}]um 224.20um \
             [expr {$c+0.20}]um 224.60um
  sky130::via3_draw
}
save
```

Why precision matters

Our first bulk loop used slightly inaccurate `uio_oe` centers and caused 21 Metal4 spacing error tiles. The corrected loop above uses exact edges. Never reuse these x-coordinates for another DEF/shuttle without examining its pin labels.

10.5 Confirm the 24 ground ties

MAGIC TKCON %

```
drc check
drc catchup
drc find 1
extract all
ext2spice lvs
ext2spice
```

You should see deliberate VGND short warnings for all 24 output-type ports and no independent `uo_out/uio_out/uio_oe` ports remaining in the extracted .subckt. The physical LEF must still retain those external pin shapes; extraction merges nodes but does not delete template pins.

11 Final DRC: how we fixed real layout errors

At this stage our centered final wrapper returned `There are no errors in tt_um_panteloukos_inverter.` This was achieved after resolving actual rule violations, not by disabling checks.

Error from our lab	Cause	Correction
met1.2: spacing < 0.14 µm	VIN and VOUT via1 Metal1 enclosures only about 0.12 µm apart	Shift VIN via1 slightly right; keep adequate Metal1 overlap
met3.6: area < 0.24 µm²	Small Metal3 via landing at ua[0]/ua[1]	Add local 0.50 × 0.50 µm Metal3 pad
met4.2: spacing < 0.30 µm	Top-edge pin neck misaligned to original pin x-position	Redraw each small neck with exact pin width
Unexpected VDD–VGND short	via3_draw inherited oversized selection box	Rebuild from known-good copy and use tightly set via boxes

11.1 The final DRC protocol

MAGIC TKCON %

```
save
drc check
drc catchup
drc find 1
drc why
```

When `drc find 1` says no errors and `drc why` says no errors, the local source layout is DRC clean under the Magic SKY130 rules loaded. If `drc count` prints a confusing total after listing error tiles, do not interpret that total as an all-clear: rely on `drc find 1` and any error boxes.

11.2 Final extraction result to save in the lab notebook

MAGIC TKCON %

```
extract all
ext2spice lvs
ext2spice
```

LINUX TERMINAL $

```
cd /foss/designs/tt_inverter/mag
grep -n -A1 -i "Xinv_layout_clean" \
  tt_um_panteloukos_inverter.spice
head -25 tt_um_panteloukos_inverter.spice
```

Record the exact child line, DRC message and any intentional VGND-short warnings as evidence. Good engineering notes record the result of a check, not just the commands used to run it.

12 Generate final LEF and GDS

12.1 What LEF and GDS mean

Format	Contains	Purpose
LEF	Macro boundary, pin names/rectangles/layers, obstructions	Abstract interface for integration
GDSII	Actual wells, diffusion, poly, contacts, metal shapes, hierarchy	Physical fabrication layout database
MAG	Editable Magic-specific hierarchical source	Your working layout database

The LEF is not a picture of the transistors. The GDS is the full geometric output. KLayout normally opens the GDS; Magic can read and write both source MAG and exported GDS.

12.2 Confirm metadata and Verilog black-box name

Edit the cloned `info.yaml` so it has `top_module: "tt_um_panteloukos_inverter"`, `tiles: "1x2"`, `analog_pins: 2`, `uses_vapwr: false`; describe `ua[0]` as VIN and `ua[1]` as VOUT. Keep unused pinout descriptions blank. Change `src/project.v` module name to exactly match.

LINUX TERMINAL $

```
cd /foss/designs/tt_inverter
grep -nE "top_module:|tiles:|analog_pins:|uses_vapwr:" info.yaml
grep -nE "ua\[0\]|ua\[1\]" info.yaml
grep -n "module tt_um_" src/project.v
mkdir -p gds lef
```

The Verilog file is a **black-box interface** for analog integration. The real circuit is implemented in the GDS. In this lab, unused outputs were tied to ground **in the layout**, not by adding functional Verilog gate logic.

12.3 Export LEF with -pinonly and GDS from Magic

MAGIC TKCON %

```
save
lef write ../lef/tt_um_panteloukos_inverter.lef -pinonly
gds write ../gds/tt_um_panteloukos_inverter.gds
```

LINUX TERMINAL $

```
cd /foss/designs/tt_inverter
ls -lh lef/tt_um_panteloukos_inverter.lef \
       gds/tt_um_panteloukos_inverter.gds
head -60 lef/tt_um_panteloukos_inverter.lef
```

We generated a 16 KB LEF and a 24 KB GDS. File sizes are merely sanity signals; correctness comes from the pin table, DRC and re-imported connectivity checks.

12.4 Verify real pins and forbidden layers

LINUX TERMINAL $

```
grep -n -A10 -E "^  PIN (VDPWR|VGND)$" \
  lef/tt_um_panteloukos_inverter.lef
grep "^  PIN " lef/tt_um_panteloukos_inverter.lef
grep -R -n "^<< metal5 >>" mag/*.mag
grep -R -n "^<< via4 >>" mag/*.mag
```

Our LEF contained `SIZE 161.000 BY 225.760`, `ua[0]` and `ua[1]` in their fixed bottom-edge locations, and vertical VDPWR/VGND on Metal4. The two final `grep` searches found no Metal5 or via4 in the Magic hierarchy.

13 KLayout and independent GDS round-trip inspection

13.1 Open the actual final GDS in KLayout

LINUX TERMINAL $

```
cd /foss/designs/tt_inverter
klayout gds/tt_um_panteloukos_inverter.gds
```

In KLayout, start at the `tt_um_panteloukos_inverter` top cell. Use Fit/Zoom Fit to inspect the 161 × 225.76 µm macro. Turn layers on and off in the Layers panel. Expand the hierarchy or open `inv_layout_clean` as the displayed cell to inspect the PMOS/NMOS instead of only their instance outline.



Figure 5 — KLayout showing inv_layout_clean as the active cell, with visible hierarchy children and local transistor geometry. This is what the earlier “empty rectangle” expands into.

The wrapper looks sparse because it contains a two-transistor inverter. It is not supposed to resemble a standard-cell-based digital GDS packed with rows, tap/filler cells and routing. Visual density is not a correctness criterion.

13.2 What to inspect visually

- The two long Metal4 rails are at the left; the analog ua pins remain on the bottom edge and the unused outputs are grounded at the top.
- The inverter instance exists and its PMOS/NMOS geometry can be viewed by increasing hierarchy depth or making inv_layout_clean the displayed cell.
- No obvious shape protrudes past the defined macro boundary; no Metal5 or via4 has been used by the custom design.
- The KLayout hierarchy shows both the inverter macro and its transistor child cells.

13.3 Re-import the GDS into a new Magic directory

This is an independent check of the **exported file**, not simply another DRC on the source .mag. Work in a separate directory so extraction files do not get mixed with the original layout.

LINUX TERMINAL $

```
mkdir -p /foss/designs/tt_inverter/gds_check
cd /foss/designs/tt_inverter/gds_check
magic -rcfile /foss/pdks/sky130A/libs.tech/magic/sky130A.magicrc
```

MAGIC TKCON % (FRESH GDS SESSION)

```
gds read ../gds/tt_um_panteloukos_inverter.gds
load tt_um_panteloukos_inverter
drc check
drc catchup
drc find 1
extract all
ext2spice lvs
ext2spice
```

LINUX TERMINAL $ (VERIFY GDS-DERIVED NETLIST)

```
cd /foss/designs/tt_inverter/gds_check
grep -n -A1 -i "Xinv_layout_clean" \
  tt_um_panteloukos_inverter.spice
```

EXPECTED GDS-DERIVED SPICE / NOT A COMMAND

```
Xinv_layout_clean_0 ua[0] ua[1] VDPWR VGND inv_layout_clean
```

We obtained that exact instance line from the **gds_check** directory, after importing the GDS and extracting again. We also obtained no Magic DRC errors on the re-imported GDS. That distinction matters: checking the .spice under mag/ would only revalidate the source layout, not the exported file.

What “round-trip PASS” actually proves

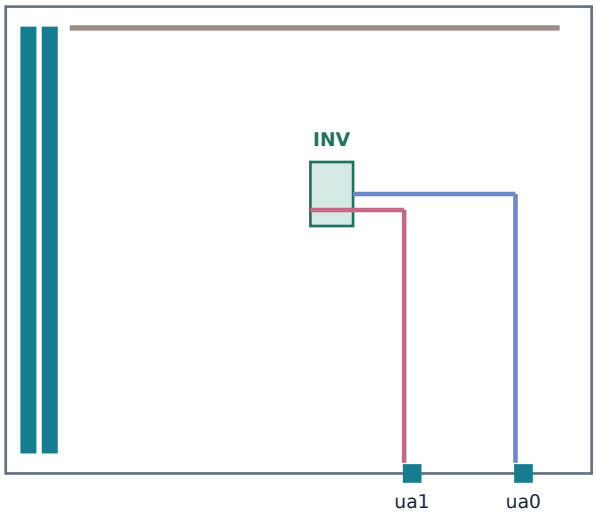
It proves that Magic can import the exported GDS, check its geometry and recover the expected top-level inverter connections. It does **not** certify that the design has passed every possible foundry or Tiny Tapeout-specific integration check.

14 Placement matters: a proposed optimization lab

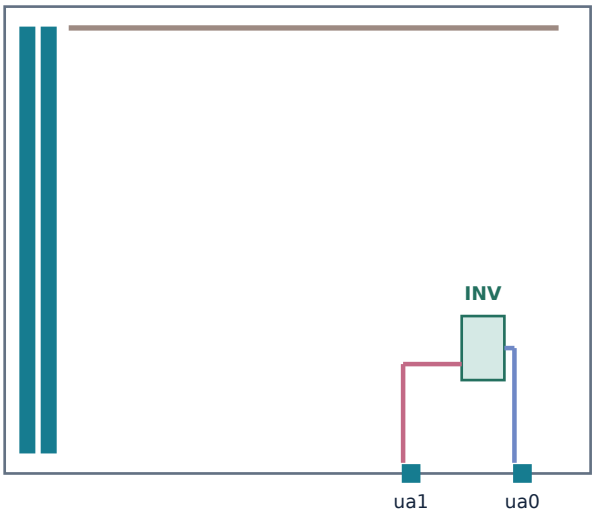
14.1 Why a DRC-clean center placement can still be poor

Our centered baseline forced the ua pins at y≈0 to reach the inverter near y≈105 µm, then travel horizontally. The long VIN/VOUT paths increase wire capacitance and potentially output delay. There was no DRC violation or incorrect logic; the drawback is **floorplanning quality**.

A VERIFIED: CENTER PLACEMENT



B PROPOSED: PIN-AWARE PLACEMENT



14.2 Better educational objective: pin-aware positioning

A sensible second exercise places the unchanged inverter hierarchy around x≈135–140 µm, y≈15–25 µm, closer to ua[1] (VOUT) and ua[0] (VIN), while preserving the power rails and the top-edge grounded-output bus. The exact optimum depends on pin placement, device well/substrate constraints, noise, power drop, and routed parasitics.

What stays the same	What must be regenerated
Standalone inverter cell; MOS sizes; 1.8 V supply; fixed TT pin geometry; grounded-output requirement	Instance transform; all local M1–M2 power and signal via coordinates; Metal2 signal routes; Metal3 power straps; then DRC/extraction/GDS
The schematic and PEX reference	New wrapper-level PEX/timing characterization is desirable before claiming a delay improvement

Do not treat proposed placement as already verified

A Magic `move` or new `getcell` position does not automatically reconnect old wires. Never reuse the centered wrapper's x/y via coordinates after relocation. Build a separate cell/version, re-extract, re-DRC and re-export.

14.3 How students can evaluate the two placements

Create two independent directories or top cells: an archival centered version and a proposed pin-aware version. For each, record DRC status, extracted mapping, route lengths, and estimated parasitics. Only then compare delay with the same external load and stimulus. This turns an imperfect initial placement into a controlled design experiment.

15 Troubleshooting reference

Symptom	What to check / do
Magic opens an empty design	Start from the directory containing the .mag; use load inv_layout_clean or give the correct path.
Instance shows only a box	Move cursor into the Magic drawing window, run expand all; in KLayout choose child cell / hierarchy depth.
“Box is not in same window as cursor”	Move pointer over the intended Magic layout window; reselect / expand there.
Large accidental via3 region	Stop, back up the broken state; rebuild from last DRC-clean snapshot; never draw a via on a long route box.
met1.2 spacing error	Inspect first DRC box; check adjacent via enclosures, not merely the original Metal1 pins.
met3.6 minimum area	Check via-stack Metal3 landing area; add a local legal patch and re-DRC.
met4.2 spacing at top outputs	Check precise pin center/edge in DEF/.mag; avoid creating a tiny Metal4 notch.
Ports VGND/uio_oe shorted warning	If these were deliberately grounded, warning is expected; check every output-type port.
Child VDD and VGND extracted as same net	Unintended short. Stop routing. Inspect box and via3 geometry; restore known-good backup.
grep finds .spice in wrong folder	Use separate mag/ and gds_check/ paths to distinguish source extraction from GDS extraction.
GDS opens sparse/empty in KLayout	Show layers, hierarchy, fit view; analog two-transistor block occupies very little macro area.
Cannot find schematic .sch	find /foss/designs/inverter -type f -name "*.sch"; open the actual path with xschem.
netgen LVS errors despite DRC	Inspect device model names, pin order, bulk connections, reference .subckt and setup.tcl.

16 Evidence-based verification checklist

Check	Result in this lab	What it does not prove
Schematic transient	PASS; inverter switches 0–1.8 V	Transistors physically laid out correctly
Standalone Magic DRC	PASS	Schematic-to-layout equivalence
Standalone Netgen LVS	PASS: Circuits match uniquely	Correct timing after routing
Standalone PEX simulation	PASS; ~8% more delay	Timing of complete Tiny Tapeout wrapper
Top-level extracted connectivity	PASS: ua[0], ua[1], VDPWR, VGND	Independent full-wrapper LVS
Grounded unused outputs	PASS: 24 intentional VGND merges	Complete shuttle precheck
Top-level Magic DRC	PASS	External precheck/foundry sign-off
LEF interface / forbidden layers	Checked; no met5/via4 in MAG	All possible integration constraints
GDS round-trip DRC	PASS	All foundry sign-off checks
GDS-derived extraction	PASS; same child instance line	Independent top-level LVS or measurement on silicon
KLayout inspection	Observed hierarchy and sparse analog layout	Circuit simulation
Official Tiny Tapeout precheck / upload	NOT DONE	Nothing has been submitted

Final local conclusion

The centered custom inverter is a well-verified **teaching baseline**: the standalone circuit matches its schematic, the Tiny Tapeout routing extracts to the intended four connections, and the exported GDS survives independent Magic DRC and connectivity checks. This does not mean a chip has been fabricated or that the official submission prechecks have passed.

Appendix A Quick command card

Open a standalone inverter in Magic

LINUX TERMINAL $

```
cd /foss/designs/inverter/mag
magic -rcfile /foss/pdks/sky130A/libs.tech/magic/sky130A.magicrc inv_layout_clean.mag
```

Open the final Tiny Tapeout .mag in Magic

LINUX TERMINAL $

```
cd /foss/designs/tt_inverter/mag
magic -rcfile /foss/pdks/sky130A/libs.tech/magic/sky130A.magicrc \
  tt_um_panteloukos_inverter.mag
```

Open the GDS in KLayout

LINUX TERMINAL $

```
cd /foss/designs/tt_inverter
klayout gds/tt_um_panteloukos_inverter.gds
```

Import the GDS in fresh Magic

LINUX TERMINAL $

```
cd /foss/designs/tt_inverter/gds_check
magic -rcfile /foss/pdks/sky130A/libs.tech/magic/sky130A.magicrc
```

MAGIC TKCON %

```
gds read ../gds/tt_um_panteloukos_inverter.gds
load tt_um_panteloukos_inverter
expand all
drc check
drc catchup
drc find 1
```

Top-level electrical extraction

MAGIC TKCON %

```
extract all
ext2spice lvs
ext2spice
```

LINUX TERMINAL $ (FROM THE CORRECT WORKING DIRECTORY)

```
grep -n -A1 "Xinv_layout_clean" tt_um_panteloukos_inverter.spice
```

Appendix B Source material and references

This student guide draws its **measured results, exact Magic coordinates, screenshots and verification messages** from the inverter lab session. Explanatory context and current Tiny Tapeout constraints were cross-checked against the public materials below. Commands involving generated device-cell names are specific to our project.

Tiny Tapeout analog specification: pin rules, allowed metals, power, template and export workflow.
https://tinytapeout.com/specs/analog/

Current SKY130 analog repository template, including info.yaml and project.v structure.
https://github.com/TinyTapeout/ttsky-analog-template

Support tools including analog DEF templates.
https://github.com/TinyTapeout/tt-support-tools

Magic port-command reference for class/use attributes.
https://github.com/RTimothyEdwards/magic/blob/master/doc/html/port.html

SKY130 Magic Tcl device and via generator definitions.
https://github.com/fossi-foundation/open-pdks/blob/main/sky130/magic/sky130.tcl

Public comparator example for additional placement and hierarchy study.
https://github.com/EduHolg/EjemploAnalog2

All screenshots embedded here originated in the interactive lab discussion. They are teaching illustrations, not claims that an external precheck or chip fabrication took place.